Retail Sales & Customer Insights: SQL End-to-End Case Study

Prepared by: **Prajakta Kumbhar**

Tools Used: MySQL • SQL Workbench

© 2025 | SQL Portfolio Project

1. INTRODUCTION:

This case study presents a complete SQL end-to-end analysis of a retail sales dataset containing Customers, Products, and Orders. It covers SQL topics from basic SELECT queries to advanced concepts including JOINs, GROUP BY, subqueries, CTEs, window functions, and views. The objective is to extract meaningful insights related to customer behavior, product performance, and overall sales trends.

1.1 Objective:

- Analyze customer demographics and purchase behavior.
- Identify best-selling products and categories.
- Calculate total revenue and customer-level spending.
- Apply SQL techniques such as JOINs, aggregations, CTEs, window functions, and views.

1.2 Dataset Overview:

This case study based on three datasets:

1. **Customers :** The customers table includes customer details such as name, city, and registration date.
2. **Products :** The *products* table contains product information including product name, category and price.
3. **Orders :** The Orders table records all purchase transactions with dates and quantities.

These datasets are linked through:

- **CustomerID (to associate orders with customers)**
- **ProductID (to associate orders with products)**

2. Dataset Schema Design

2.1 Customer Table

Column Name	Description	Data Type
CustomerID	Unique ID for each customer	INT
CustomerName	Name of the customer	VARCHAR(50)
City	City where the customer lives	VARCHAR(40)
RegistrationDate	Date customer registered	DATE

2.2 Products Table

Column Name	Description	Data Type
ProductID	Unique ID for each product	INT
ProductName	Name of the product	VARCHAR(40)
Category	Product category (Electronics, Furniture…)	VARCHAR(40)
Price	Price of the product	INT

3. Orders Table

Column Name	Description	Data Type
OrderID	Unique ID for each order	INT
CustomerID	ID of the customer who made the order	INT
ProductID	ID of the product purchased	INT
OrderDate	Date when the order was placed	DATE
Quantity	Number of units purchased	INT

3. Data Processing

- The dataset structure was analyzed to understand relationships between Customers, Products, and Orders tables.
- Data accuracy was ensured by validating CustomerID and ProductID across all tables.
- Records were reviewed for missing or incorrect values, and date formats were standardized for proper analysis.
- Tables were joined using SQL JOIN operations to combine customer, product, and order information.
- Aggregations such as SUM, COUNT, and GROUP BY were applied to calculate total sales, revenue, and customer spending.
- Advanced SQL techniques such as subqueries, CTEs, window functions, and views were applied to generate meaningful insights.
- The processed data was structured into meaningful result sets ready for reporting and business insights.

Case Study Questions

BASIC LEVEL # # #

-- 1. Select all customers from the Customers table.

```
SELECT
  *
FROM
   Customers;
```

-- 2. Show names of all products in the Products table.

```
SELECT
    ProductName
FROM
    Products;
```

-- 3. Retrieve all orders made by CustomerID = 1.

```
SELECT
    *
FROM
    Orders
WHERE
    CustomerID = 1;
```

-- 4. Display unique product categories.

```
SELECT DISTINCT
     (Category) AS Unique_category
FROM
     Products;
```

-- 5. List customers who live in Mumbai.

```
SELECT
   *
FROM
   Customers
WHERE
    City = 'Mumbai';
```

-- 6. Show orders placed after ‘2023-07-05’.

```
SELECT
    *
FROM
    Orders
WHERE
    OrderDate > '2023-07-05';
```

-- 7. Show product details for ProductID = 103.

```
SELECT
    *
FROM
    Products
WHERE
    ProductID = 103;
```

-- 8. Count total number of customers.

```
SELECT
    COUNT(CustomerID) AS Total_Customers
FROM
    Customers;
```

-- 9. Count total number of products.

```
SELECT
    COUNT(ProductID) AS Total_Products
FROM
    Products;
```

-- 10. Get total quantity ordered in all orders.

```
SELECT
    SUM(Quantity) AS Total_Quantity
FROM
    Orders;
```

-- 11. Sort all customers by RegistrationDate.

```
SELECT
    *
FROM
    Customers
ORDER BY RagistrationDate;
```

-- 12. Show all orders sorted by OrderDate (latest first).

```
SELECT
    *
FROM
    Orders
ORDER BY OrderDate DESC;
```

-- 13. Display all products costing more than 10000.

```
SELECT
   *
FROM
   Products
WHERE
    Price > 10000;
```

-- 14. Retrieve customers who registered in 2023.

```
SELECT
    *
FROM
   Customers
WHERE
     YEAR(RagistrationDate) = 2023;
```

-- 15. Display orders where quantity is greater than 1.

```
SELECT
   *
FROM
   Orders
WHERE
   Quantity > 1;
```

Medium Level # # #

-- 16. Show customer name along with their order details (JOIN).

```
SELECT
    c.CustomerName,
    o.OrderID,
    p.ProductName,
    o.OrderDate,
    o.Quantity
FROM
    Customers c
        JOIN
    Orders o ON c.CustomerID = o.CustomerID
        JOIN
    Products p ON p.ProductID = o.ProductID;
```

-- 17. Show product name and total quantity sold for each product.

```
SELECT
    p.ProductName, SUM(o.Quantity) AS Total_Quantity
FROM
    Products p
        JOIN
    Orders o USING (ProductID)
GROUP BY p.ProductName;
```

-- 18. List customers who bought Electronics category products.

```
SELECT DISTINCT
    c.CustomerName, p.Category
FROM
    Customers c
        LEFT JOIN
    Orders o USING (CustomerID)
        JOIN
    Products p ON p.ProductID = o.ProductID
WHERE
    p.Category = 'Electronics';
```

-- 19. Calculate the total revenue (Price × Quantity) for each order.

```
SELECT
    p.ProductName,
    ROUND(SUM(p.Price * o.Quantity), 2) AS Total_Revenue
FROM
    Products p
        JOIN
    Orders o USING (ProductID)
GROUP BY  p.ProductName;
```

-- 20. Show total spending by each customer.

```
SELECT
    c.CustomerName,
    ROUND(SUM(p.Price * o.Quantity), 2) AS Total_spend
FROM
    Customers c
        JOIN
    Orders o USING (CustomerID)
        JOIN
    Products p ON p.ProductID = o.ProductID
GROUP BY c.CustomerName;
```

-- 21. Count how many orders each customer has placed.

```
SELECT
    c.CustomerName, SUM(o.Quantity) AS Total_oder_placed
FROM
    Customers c
        JOIN
    Orders o USING (CustomerID)
GROUP BY c.CustomerName;
```

-- 22. Show best-selling product (highest total quantity).

```
SELECT
    p.ProductName, SUM(Quantity) AS Total_Quantity
FROM
    Products p
        JOIN
    Orders o USING (ProductID)
GROUP BY p.ProductName
ORDER BY Total_Quantity DESC
LIMIT 1;
```

-- 23. Show number of orders for each product category.

```
SELECT
    p.Category, SUM(Quantity) AS Tota_Order
FROM
    Products p
        JOIN
    Orders o USING (ProductID)
GROUP BY p.Category;
```

-- 24. Show customers who have not placed any orders.

```
SELECT
    c.CustomerName
FROM
    Customers c
        LEFT JOIN
    Orders o USING (CustomerID)
WHERE
    o.OrderID IS NULL;
```

-- 25. Find the most expensive product.

```
SELECT
    ProductName, Price
FROM
    Products
ORDER BY Price DESC
LIMIT 1;
```

-- 26. Find the least expensive product.

```
SELECT
    ProductName, Price
FROM
    Products
ORDER BY Price
LIMIT 1;
```

-- 27. Show orders placed between '2023-07-01' and '2023-07-10'.

```
SELECT
    *
FROM
    Orders
WHERE
    OrderDate BETWEEN '2023-07-01' AND '2023-07-10';
```

-- 28. List all orders along with product price and total bill amount.

```
SELECT
    o.OrderID,
    p.ProductName,
    p.Price,
    o.Quantity,
    p.Price * o.Quantity AS Total_bill
FROM
    Products p
        JOIN
    Orders o USING (ProductID);
```

-- 29. Show products that were never ordered.

```
SELECT
   ProductName
FROM
   Products
WHERE
   ProductID NOT IN (SELECT
         ProductID
      FROM
         Orders);
```

-- 30. Count how many customers registered city-wise.

```
SELECT
   City, COUNT(CustomerID) AS Total_customer
FROM
   Customers
GROUP BY City;
```

-- 31. Show customers who placed an order on the same date as CustomerID = 3.

```
SELECT DISTINCT
   c.CustomerName, o.OrderDate
FROM
   Customers c
      JOIN
   Orders o USING (CustomerID)
WHERE
   OrderDate IN (SELECT
         OrderDate
      FROM
         Orders
      WHERE
         CustomerID = 3)
      AND c.CustomerID <> 3;
```

-- 32. Retrieve customers who bought the most expensive product.

```
SELECT
   c.CustomerName, p.ProductName, p.Price
FROM
   Customers c
      JOIN
   Orders o USING (CustomerID)
      JOIN
   Products p USING (ProductID)
WHERE
   p.Price = (SELECT
         MAX(Price)
      FROM  Products);
```

-- 33. List customers who have placed more orders than the average number of orders per customer.

```
SELECT
   c.CustomerName, COUNT(o.OrderID) AS Order_placed
FROM
   Customers c
      JOIN
   Orders o USING (CustomerID)
GROUP BY c.CustomerName
HAVING Order_placed > (SELECT
      AVG(Order_count)
   FROM
      (SELECT
         COUNT(OrderID) AS Order_count
      FROM
         Orders
      GROUP BY CustomerID) AS t);
```

-- 34. Find products whose price is higher than every product in Accessories category.

```
SELECT
   ProductName, Price
FROM
   Products
WHERE
   Price > ALL (SELECT
         Price
      FROM
         Products
      WHERE
         Category = 'Accessories');
```

-- 35. Retrieve customers whose total purchase amount is higher than the average customer spending.

```
SELECT
   c.CustomerName, SUM(p.Price * o.Quantity) AS Total_Purchase
FROM
   Customers c
      JOIN
   Orders o USING (CustomerID)
      JOIN
   Products p USING (ProductID)
GROUP BY c.CustomerName
HAVING Total_Purchase > (SELECT
      AVG(Cust_spend)
   FROM
      (SELECT
         SUM(p.Price * o.Quantity) AS Cust_spend
      FROM
         Products
            JOIN Orders o USING (ProductID)
        GROUP BY o.CustomerID) AS C);
```

Advanced Level # # #

-- 36. Show top 3 customers by total spending.

```
SELECT
   c.CustomerName, SUM(p.Price * o.Quantity) AS Total_Spend
FROM
   Customers c
      JOIN
   Orders o USING (CustomerID)
      JOIN
   Products p USING (ProductID)
GROUP BY c.CustomerName
LIMIT 3;
```

-- 37. Show month-wise total sales revenue.

```
SELECT
   MONTHNAME(o.OrderDate) AS Months,
   ROUND(SUM(p.Price * o.Quantity), 2) AS Total_revenue
FROM
   Orders o
      JOIN
   Products p USING (ProductID)
GROUP BY Months;
```

-- 38. Display customers who purchased more than 1 category of products.

```
SELECT
   c.CustomerName, COUNT(DISTINCT p.Category) AS CategoryCount
FROM
   Customers c
      JOIN
   Orders o USING (CustomerID)
      JOIN
   Products p USING (ProductID)
GROUP BY c.CustomerName
HAVING CategoryCount > 1;
```

-- 39. Show products whose total revenue > 20,000.

```
SELECT
   p.ProductName, SUM(p.Price * o.Quantity) AS Total_revenue
FROM
   Products p
      JOIN
   Orders o USING (ProductID)
GROUP BY p.ProductName
HAVING Total_revenue > 20000;
```

-- 40 Create a view for customer purchase summary.

```
CREATE VIEW Cust_Purchases_summary AS
   (SELECT
      c.CustomerID,
      c.CustomerName,
      COUNT(DISTINCT o.OrderID) AS Total_orders,
      SUM(o.Quantity) AS Total_quantity,
      SUM(p.Price * o.Quantity) AS Total_revenue
   FROM
      Customers c
         LEFT JOIN
      Orders o USING (CustomerID)
         JOIN
      Products p USING (ProductID)
   GROUP BY c.CustomerID , c.CustomerName);

SELECT
   *
FROM
   Cust_Purchases_summary;
```

-- 41. Use a CTE to calculate total revenue for each category.

```
WITH Total_revenue AS(SELECT p.Category,SUM(p.Price*o.Quantity) AS Total
                              FROM Products p JOIN Orders o
                   USING(ProductID)
                    GROUP BY Category)
SELECT *FROM Total_revenue;
```

-- 42. Use a Window Function to find category-wise average price.

```
SELECT DISTINCT Category,AVG(Price) OVER(PARTITION BY Category) AS Avg_price
FROM Products;
```

-- 43. Use a Window Function to rank customers by number of orders.

```
SELECT RANK() OVER(ORDER BY Total_orders) AS
Ranks,CustomerID,CustomerName,Total_orders
FROM(SELECT c.CustomerID,c.CustomerName,COUNT(o.OrderID) AS Total_orders
FROM Customers c JOIN Orders o
USING(CustomerID) GROUP BY  c.CustomerID,c.CustomerName)AS T;
```

-- 44. Write a subquery to find products priced above the average product price.

```
SELECT
   ProductName, Price
FROM
   Products
     WHERE
        Price > (SELECT
              AVG(Price)
           FROM
              Products);
```

-- 45. Find products whose price is higher than every product in Accessories category.

```
SELECT
   ProductName, Price
FROM
   Products
WHERE
   Price > ALL (SELECT
         Price
       FROM
         Products
       WHERE
         Category = 'Accessories');
```

---

4. Business Insights & Analysis

Based on the queries, we can derive the following insights:

- Electronics category contributes the highest share of total revenue
- Identified top-spending customers driving maximum sales
- Headphones and mobile products show consistent demand
- Sales peak observed during July 2023 transactions
- Customers purchasing across multiple categories generate higher revenue

5. Conclusion

This SQL end-to-end case study successfully analyzed retail sales data to uncover customer behavior, product performance, and revenue trends. By applying SQL techniques such as joins, aggregations, subqueries, CTEs, window functions, and views, meaningful business insights were generated. The analysis demonstrates how SQL can be effectively used for data-driven decision-making.

6. Recommendations

- Focus more on electronics products because they sell the most.
- Take care of top customers by giving offers and rewards.
- Keep enough stock of popular products like mobiles and headphones.
- Run offers during busy sales months to increase sales.
- Encourage customers to buy more than one type of product.

7. Future Scope

- Use advanced analytics and forecasting to predict future sales and demand.
- Build interactive dashboards using Power BI or Tableau for real-time business insights.
- Expand the dataset with customer behavior, discounts, and returns for deeper analysis.